Mathematical Foundations of Data Sciences

Gabriel Peyré
CNRS & DMA, École Normale Supérieure

CHAPTER 3

Shannon Coding Theory

September 5, 2026

A sequence of symbols can often be stored using fewer bits when its probabilities and dependencies are taken into account. The challenge is to reduce the expected length while keeping every message unambiguously recoverable. Prefix codes and entropy lead to Shannon's source coding bound; Huffman's construction, block coding, and models of dependent symbols then show how coding methods approach that bound.

3.1 Source Coding

Uniform coding. We consider an alphabet $(s_1, \dots, s_K)$ of K symbols. For signal samples $0 \leqslant f_0 < 1$ quantized into K bins, the symbols label the bins. For text, they may represent letters, punctuation marks, and spaces. A fixed-length binary code uses $\lceil \log_2(K) \rceil$ bits per symbol. For example, if $K = 4$ and the symbols are $\{0, 1, 2, 3\}$, one possible code is $(c_0 = 00, c_1 = 01, c_2 = 10, c_3 = 11)$.

When some symbols are much more frequent than others, variable-length codes can reduce the average number of bits per symbol by assigning shorter codewords to frequent symbols.

Prefix coding. A binary code assigns each symbol s_k a finite word $c_k = c(s_k) \in \bigcup_{\ell \geqslant 0} \{0, 1\}^\ell$ of length $|c_k|$. An empty word is allowed only for a one-symbol alphabet, with the message length supplied separately. A code is a prefix code if no codeword is a prefix of another. Equivalently, the codewords correspond to leaves of a binary tree T: starting at the root, traversing an edge to the left appends 0, and traversing an edge to the right appends 1. We write $c = \text{Leaves}(T)$ for this tree representation.

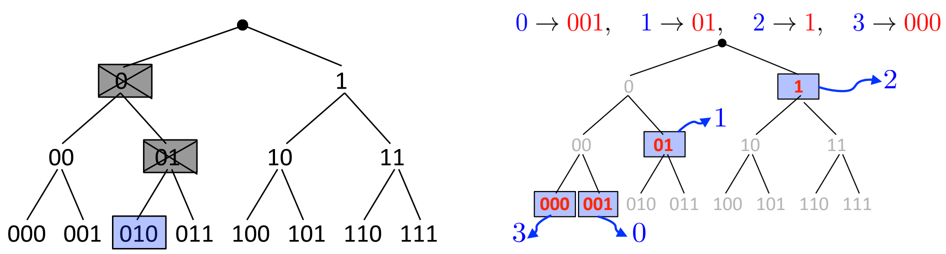

Figure 3.1. Left: the complete tree of binary words of length 3. Right: a prefix code.

To decode a binary stream, follow its bits down the tree. Upon reaching a leaf, output the corresponding symbol and return to the root. The prefix property makes this decoding unambiguous.

Probabilistic modeling. We seek a code with the smallest expected length. Let $p = (p_1, \dots, p_K) \in \mathbb{R}_+^K$ be the symbol probability vector, with $\sum_k p_k = 1$. In practice, one often estimates p_k from the relative frequency of s_k in the data. The decoder must also know the resulting code, either from a shared model or from a transmitted description.

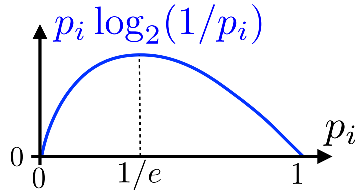

Figure 3.2. The concave function $h(u) = -u \log_2 u$ underlying entropy.

The entropy of the probability distribution p is

$$H(p) \stackrel{\text{def.}}{=} -\sum_k p_k \log_2(p_k)$$

with the convention $0 \log_2(0) = 0$.

For $h(u) = -u \log_2 u$ and $u > 0$, one has $h'(u) = -(\ln u + 1)/\ln 2$ and $h''(u) = -1/(u \ln 2) < 0$. Thus H is strictly concave on the probability simplex.

For a probability density f with respect to a reference measure $\mathrm{d}x$, such as Lebesgue measure on $\mathbb{R}^d$, differential entropy is defined by $H(f) \stackrel{\text{def.}}{=} -\int f(x) \log_2(f(x))\,\mathrm{d}x$, whenever this integral is defined. Unlike discrete entropy, it can be negative and depends on the reference measure.

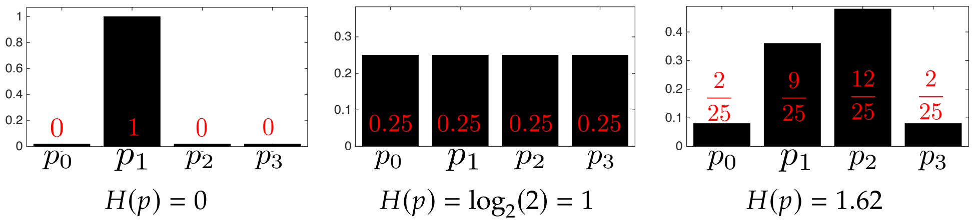

Figure 3.3. Three examples of probability distributions with corresponding entropies.

Lemma 3.1. *For every probability vector p,*

$$0 = H(\delta_i) \leqslant H(p) \leqslant H(1/K) = \log_2(K)$$

where δ_i is the point mass at symbol i, and $1/K$ denotes the uniform probability vector.

Proof. For $u \in [0,1]$, $-u \log_2 u \geqslant 0$, so $H(p) \geqslant 0$, with equality precisely for a point mass. For probability vectors p, q with $q_i > 0$ whenever $p_i > 0$, the inequality $\ln u \leqslant u - 1$ gives

$$\sum_{i:p_i>0} p_i \log_2 \frac{q_i}{p_i} \leqslant \frac{1}{\ln 2}\left(\sum_{i:p_i>0} q_i - 1\right) \leqslant 0.$$

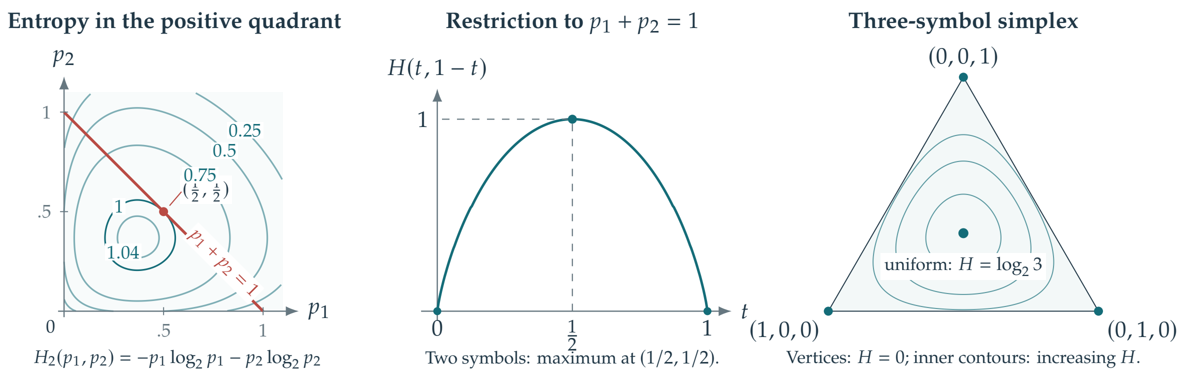

Figure 3.4. Entropy is minimized at a point mass and maximized at the uniform distribution. Left: level sets of the entropy formula in the positive quadrant and the constraint $p_1 + p_2 = 1$. Middle: its restriction to that constraint. Right: entropy level sets on the three-symbol probability simplex.

Consequently $H(p) \leqslant -\sum_i p_i \log_2 q_i$. Taking $q_i = 1/K$ yields $H(p) \leqslant \log_2 K$, with equality precisely when p is uniform. □

The quantity $\mathrm{KL}(p \mid q) \overset{\text{def.}}{=} \sum_i p_i \log_2(p_i/q_i) \geqslant 0$ is called the Kullback–Leibler divergence, or relative entropy between p and q. The conventions are $0 \log_2(0/q_i) = 0$ and $p_i \log_2(p_i/0) = +\infty$ for $p_i > 0$. Although it vanishes exactly when $p = q$, it is not a metric: it is generally asymmetric and does not satisfy the triangle inequality. Minimizing the cross-entropy between an empirical distribution and a model is equivalent to maximizing the likelihood, and to minimizing their relative entropy. Cross-entropy is therefore a common objective in classification.

3.2 Shannon Source Coding Theorem

For a symbol distribution p, the expected codeword length of a code c is

$$L(c) \overset{\text{def.}}{=} \sum_k p_k |c_k|.$$

We seek a prefix code minimizing $L(c)$. Shannon's source coding theorem bounds the optimal expected length in terms of entropy.

Theorem 3.2. *Assume $p_k > 0$ for every symbol in the alphabet. Symbols of zero probability may be omitted. (i) If $c = \mathrm{Leaves}(T)$ for some tree T, then*

$$L(c) \geqslant H(p).$$

(ii) Conversely, there exists a code c with $c = \mathrm{Leaves}(T)$ such that

$$L(c) < H(p) + 1.$$

Kraft's inequality characterizes the possible codeword lengths of a prefix code and is the key to the proof.

Lemma 3.3 (Kraft inequality)**.** *(i) For a code c, if there exists a tree T such that $c = \mathrm{Leaves}(T)$ then*

$$\sum_k 2^{-|c_k|} \leqslant 1. \tag{3.1}$$

(ii) Conversely, if $(\ell_k)_k$ are nonnegative integers such that

$$\sum_k 2^{-\ell_k} \leqslant 1 \tag{3.2}$$

then there exists a code $c = \mathrm{Leaves}(T)$ such that $|c_k| = \ell_k$.

Proof. $\Rightarrow$ Suppose $c = \mathrm{Leaves}(T)$ and put $m = \max_k |c_k|$. Complete T to a full binary tree of depth m. The subtree rooted at c_k has height $m - |c_k|$ and $2^{m-|c_k|}$ leaves; see Figure 3.5, panel 1. These subtrees are disjoint, and the full tree has 2^m leaves, so

$$\sum_k 2^{m-|c_k|} \leqslant 2^m,$$

which gives (3.1) after division by 2^m.

$\Leftarrow$ Conversely, assume (3.2) holds and order the lengths so that $\ell_1 \leqslant \ldots \leqslant \ell_K =: m$. We construct full subtrees of height $m - \ell_k$, each with $2^{m-\ell_k}$ leaves, and place their leaf blocks consecutively from left to right. The inequality $\sum_k 2^{m-\ell_k} \leqslant 2^m$ ensures that all blocks fit among the 2^m leaves. Because the lengths ℓ_k are nondecreasing, the block sizes are nonincreasing powers of two. Each block therefore begins at a multiple of its own size and is the leaf set of a subtree; see Figure 3.5, panel 2. The roots of these disjoint subtrees form the required prefix code. □

Proof of Shannon's theorem. (i) Put $\ell_k = |c_k|$, $Z = \sum_k 2^{-\ell_k} \leqslant 1$, and $q_k = 2^{-\ell_k}/Z$. The nonnegativity of relative entropy gives

$$L(c) - H(p) = \mathrm{KL}(p \mid q) - \log_2 Z \geqslant 0. \tag{3.3}$$

Equality requires $Z = 1$ and $q = p$, identifying the ideal real-valued code lengths $-\log_2 p_k$.

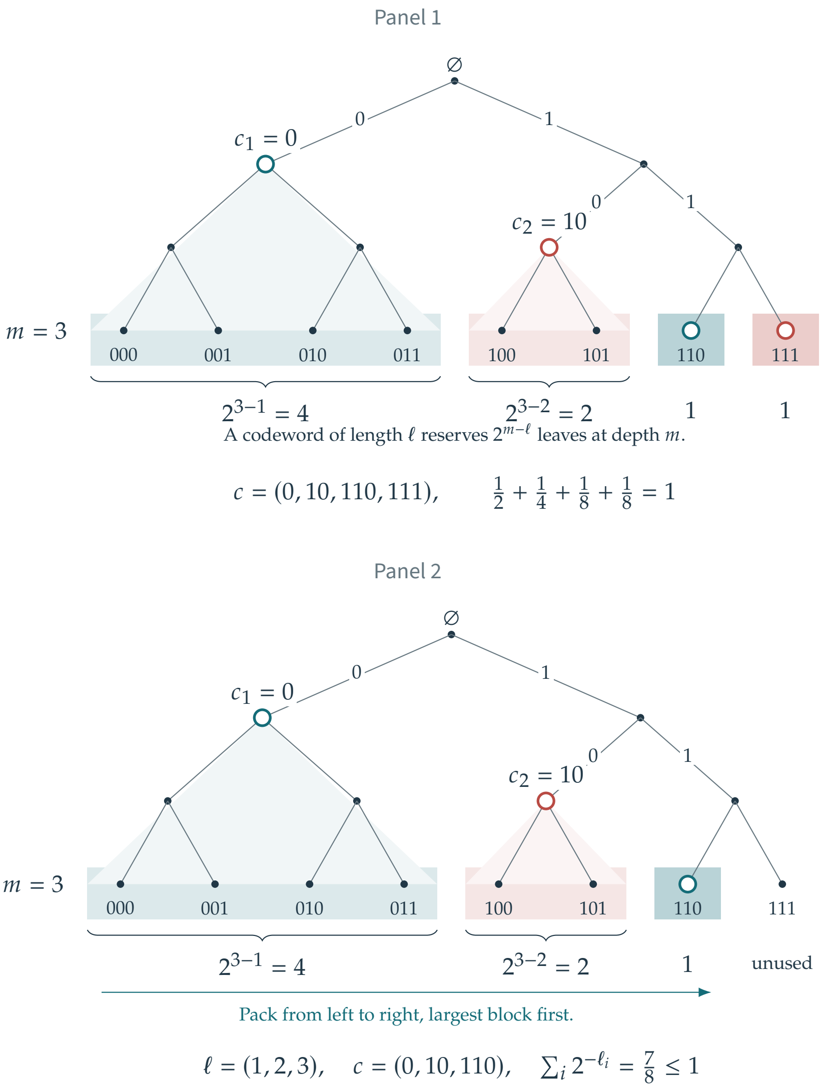

Figure 3.5. Panel 1: full binary tree obtained by completing the tree associated with the code ($c_1 = 0, c_2 = 10, c_3 = 110, c_4 = 111$). Panel 2: subtrees with the prescribed codeword lengths packed from left to right.

(ii) Choose $\ell_k = \lceil -\log_2 p_k \rceil$. Then $\sum_k 2^{-\ell_k} \leqslant \sum_k p_k = 1$, so Lemma 3.3 constructs a prefix code with these lengths. Moreover,

$$L(c) = \sum_k p_k \lceil -\log_2 p_k \rceil < \sum_k p_k(-\log_2 p_k + 1) = H(p) + 1.$$

For a one-symbol alphabet, the empty word has length zero and the same conclusions hold. □

This proof is constructive: rounding the ideal lengths and using Kraft's lemma yields a Shannon code. It need not minimize $L(c)$. Huffman's greedy algorithm constructs an optimal binary prefix code by repeatedly merging the two least probable symbols. With a priority queue, its cost is $O(K \log K)$. Figure 3.6 illustrates the construction.

`Associated code: coding/test_text.m`

3.3 Probabilistic Modeling

Symbol-by-symbol prefix coding may be inefficient when one symbol has probability close to one: its ideal length $-\log_2 p_k$ is then below one bit, whereas a nontrivial prefix code uses at least one bit per symbol. A remedy is to group r consecutive symbols into blocks and code the resulting alphabet of size K^r.

For independent symbols with common distribution p, the block distribution is

$$P_{k_1,\ldots,k_r} = p_{k_1} \cdots p_{k_r}, \qquad H(P) = rH(p).$$

Shannon's theorem applied to blocks gives an expected length per original symbol in the interval $[H(p), H(p) + 1/r)$. This is an exact expectation bound under the model; no large-sample limit is needed. The code is prefix-free on blocks, so it need not use an integer number of bits per original symbol on average.

For dependent symbols whose marginal distributions are all p, entropy subadditivity gives

$$H(P) \leqslant rH(p),$$

with equality if and only if the symbols within the block are independent. For N divisible by r, the expected total length is at most

$$\frac{N}{r}\left(H(P) + 1\right) \leqslant N\left(H(p) + \frac{1}{r}\right).$$

Modeling dependence can therefore reduce the expected length further. For a stationary source on a finite alphabet, the block entropies are subadditive, so the entropy rate $\lim_{r\to\infty} H(P)/r$ exists. This rate is the asymptotic lower bound on the expected number of bits per symbol.

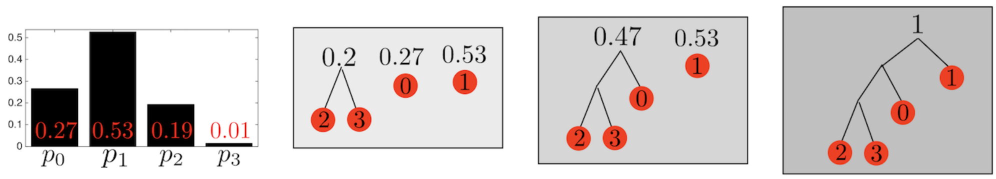

Figure 3.6. Successive merges in Huffman's coding algorithm.

Mismatched probability models. For ideal real-valued lengths based on a model q, the excess expected length is the relative entropy, also called the Kullback–Leibler divergence. For $q_k > 0$ whenever $p_k > 0$, this excess is

$$\mathrm{KL}(p|q) = -\sum_k p_k \log_2 q_k - H(p) = \sum_k p_k \log_2 \frac{p_k}{q_k} \geqslant 0.$$

After lengths are rounded to integers, the difference between the expected lengths of two prefix codes need not equal this divergence. Relative entropy is jointly convex in (p, q) and is a central tool for comparing probability distributions in information theory and statistics.

3.4 Exploiting Statistical Dependence

A memoryless code based on the marginal distribution cannot have expected length below the marginal entropy. Dependent data, however, can be coded more efficiently by modeling their joint distribution. An invertible transformation of the discrete symbols can make the dependence easier to encode while preserving their joint entropy. For example, consecutive samples of a smooth signal often have small differences. Figure 3.7 illustrates coding these differences instead of the original values. The initial sample must also be retained to make the transformation invertible.

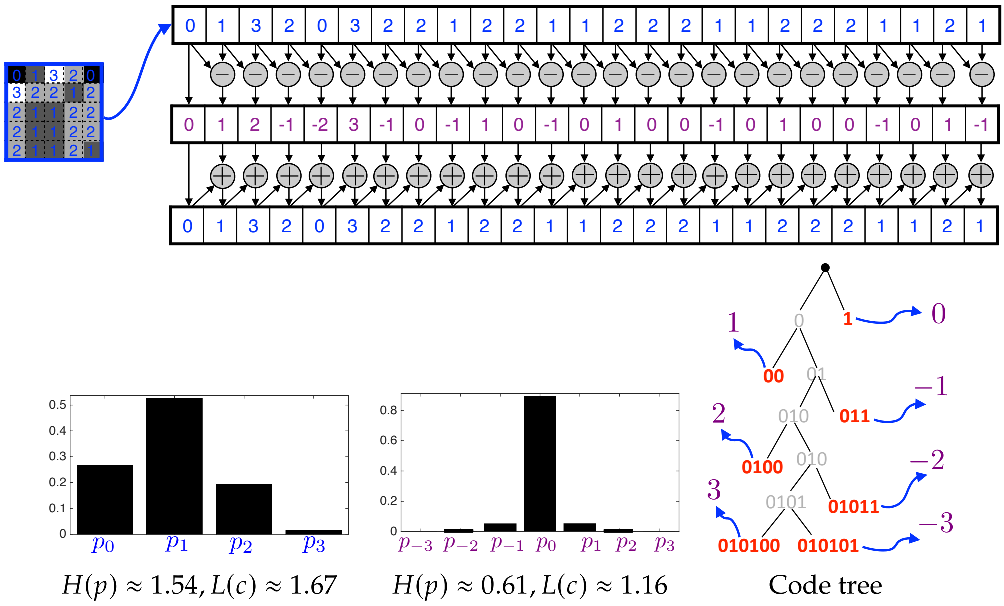

Figure 3.7. Top: transformation by successive differences. Bottom: comparison of histograms of pixel values and differences, and a code tree for these differences.

Another way to exploit temporal redundancy is run-length coding: each maximal run of identical symbols is represented by the symbol and the run length, which can itself be entropy-coded. Its performance depends on how the run lengths and transitions are modeled.

For a stationary finite-state Markov chain with stationary probabilities π_i and transition probabilities P_{ij}, the entropy rate is $-\sum_{i,j} \pi_i P_{ij} \log_2 P_{ij}$. Coding with the conditional transition probabilities, or coding sufficiently long blocks, can approach this rate. Run-length coding alone does not guarantee this performance.